

Taller de programación I
Catedra: Deymonnaz
2C 2025

Proyecto RoomRTC

Grupo Cargo meet

Del Pup, Federico - 108768
Sanjines, Aaron - 99176
Zugna, Federico - 95758

Índice

1. Introducción **4**
2. Objetivo del proyecto **4**
3. Arquitectura del Sistema **5**
3.1. Diagrama General 5
3.2. Componentes 6
3.2.1. Signaling Server (modelo Cliente–Servidor) 6
3.2.2. Cliente (UI + Media + Red) 7
3.3. flujo de ejecución 7
A. UI (Iced) + canales internos 7
B. Señalización (TLS) en background 7
C. ICE-lite / transporte UDP base (socket único) 7
D. Canal seguro: DTLS + verificación de fingerprint 8
E. Video: VP8 + (RTP-Lite) + Jitter Buffer 8
F. Cierre ordenado (shutdown) 8
4. Diseño e Implementación **9**
4.1. Señalización 9
4.1.1. Objetivo 9
4.1.2. Protocolo de aplicación (sobre TLS) 9
Secuencia completa de establecimiento 10
4.1.3. Parámetros de red (UDP base) 10
4.1.4. Handshake DTLS + verificación de fingerprint 10
4.2. Transmisión de video seguro 11
4.2.1. Objetivo 11
4.2.2. Pipeline: captura/encode - packetización - DTLS 11
4.2.3. Recepción: DTLS - demux - reensamble - jitter buffer 12
4.2.4. Control adaptativo simple 12
4.2.5. Consideraciones de concurrencia 12
4.3. Sincronización y Jitter Buffer 13
4.3.1. Problema a Resolver 13
4.3.2. Estrategia implementada 13
4.3.3. Consumidor de Jitter 13
5. Diseño e implementación del agregado final **14**
5.1. Transferencia de Archivos 14
5.1.1. Objetivo 14
5.1.2. Diseño: SCTP sobre DTLS 14
5.1.3. Canal logico: Stream SCTP dedicado 14
5.1.4. Formato de aplicación del archivo 14
5.1.5. Envío 14
5.1.6. Recepción 15
5.2. Audio 15
5.2.1. Objetivo 15
5.2.2. Diseño: Opus/RTP por UDP 15

5.2.3. Pipeline Gstreamer 15
6. Modelo de concurrencia 16
6.1. Hilo Principal (Main / GUI) 16
6.2. Hilo de Señalización (`auth_thread`) 16
6.3. Hilo Gestor de DTLS (`DTLS Manager`) 16
6.4.1. Hilo de Transmisión (`tx_thread`) 17
6.4.2. Hilo de Recepción (`rx_thread`) 17
6.4.3. Hilo Consumidor de Jitter Buffer (`jitter_thread`) 17
6.5. Hilos de Datos (SCTP) 18
6.5.1. Hilo de Envío SCTP (`sctp_tx_thread`) 18
6.6. Hilos de Audio 18
6.6.1. Hilo TX Audio (`audio_tx_thread`) 18
6.6.2. Hilo RX Audio (`audio_rx_thread`) 19
6.7. Hilos Auxiliares 19
7. Conclusiones 19

1. Introducción

Desde la pandemia de COVID-19 en 2020, el software para videoconferencias cobra una importancia central, ya que posibilitó la realización de clases, congresos y diversos eventos de manera remota. Aun hoy, 5 años después, este tipo de sistemas sigue en vigencia.

En este contexto, la Universidad de Buenos Aires desea implementar un sistema de videoconferencias para transmitir cursos, congresos, etc. Para que un software de este tipo sea considerado de calidad, se requiere que cumpla con ciertos requisitos tanto funcionales como no funcionales, como podrían ser: Usabilidad, escalabilidad, velocidad, bajo consumo de red, confiabilidad, seguridad, entre otros.

Tras analizar las diferentes alternativas existentes, tanto en software libre como alternativas privativas, se decide implementar una solución in-house.

La misma deberá contar con una implementación propia de WebRTC. Siendo esta una serie de protocolos que permiten la transmisión de video y sonido en tiempo real por parte de navegadores y aplicaciones móviles. Debido a las necesidades de eficiencia y seguridad propias de este tipo de aplicaciones, el lenguaje de implementación será Rust.

2. Objetivo del proyecto

El objetivo principal del presente Trabajo Práctico es desarrollar una versión en el lenguaje Rust del stack WebRTC, de manera tal que sea posible la realización de videoconferencias entre usuarios en distintos dispositivos. El proyecto debe respetar las specs (i.e., especificaciones), haciendo énfasis en la compatibilidad con clientes existentes de WebRTC. Aspectos no funcionales como la escalabilidad y la eficiencia de la solución deben ser contemplados, ya que el proyecto debe poder ser utilizado por un gran número de personas, desde dispositivos de diversa potencia.

El objetivo secundario es desarrollar un proyecto real de software de mediana envergadura aplicando buenas prácticas de desarrollo, incluyendo entregas y revisiones periódicas. La idea es, después de todo, que este proyecto signifique no

sólo la aplicación de los temas vistos en la asignatura, sino que, del conjunto de conocimientos aprendidos durante la carrera hasta el momento.

3. Arquitectura del Sistema

El sistema implementa una arquitectura híbrida que combina:

- **Cliente–Servidor** para la fase de señalización y control (autenticación, lobby, establecimiento/fin de llamada e intercambio de parámetros).
- **Peer-to-Peer (P2P)** para la transmisión de datos multimedia y archivos.

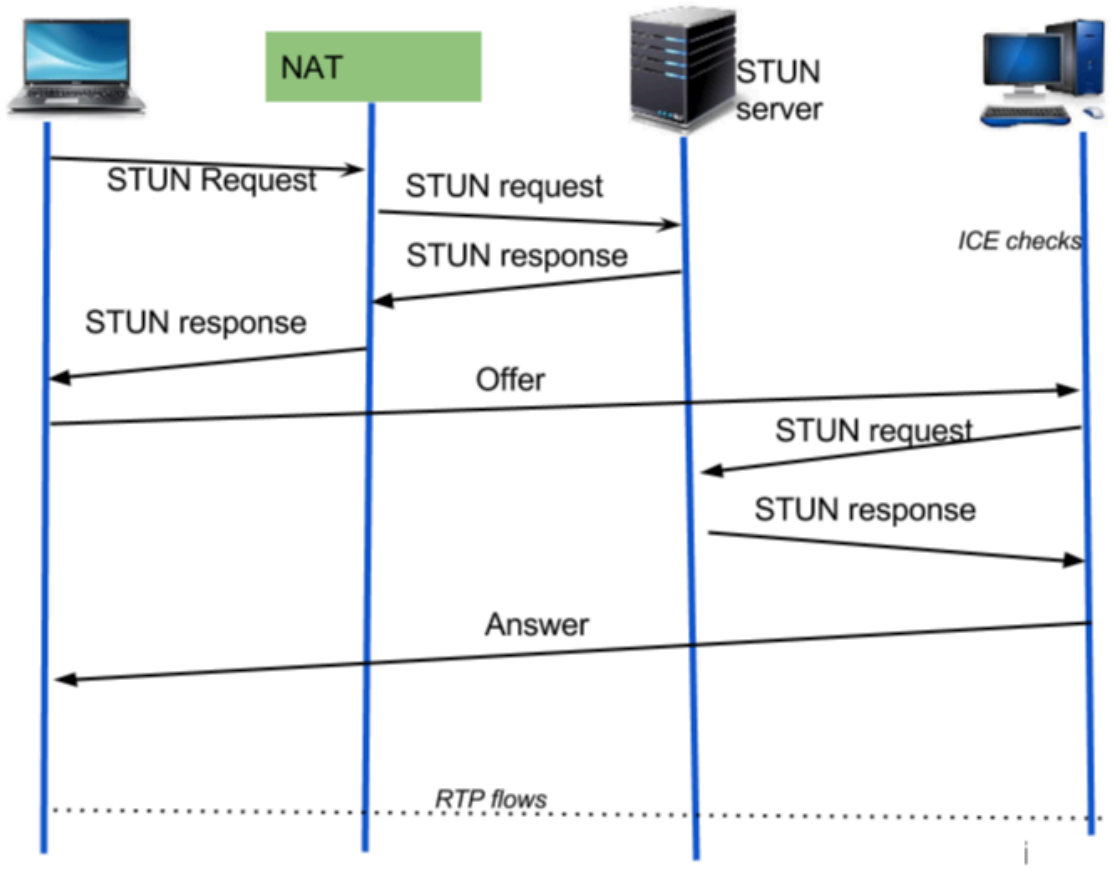
Este enfoque reduce la carga del servidor central (evitando que transporte audio/video/archivos), disminuye la latencia y permite que el ancho de banda principal circule directamente entre los participantes. El servidor queda limitado a funciones de coordinación y orquestación de la sesión.

En la solución se distinguen tres planos:

- **Plano de control (Signaling):** mensajes estructurados sobre TCP + TLS.
- **Plano de media seguro (P2P):** UDP + DTLS, donde viajan video y la transferencia de archivos (SCTP multiplexado) y reportes de control (RTCP).

3.1. Diagrama General

La Figura 3.1 resume la topología y los canales principales. El Signaling Server centraliza el control y coordina el inicio de sesión y el establecimiento de llamadas, mientras que la media y los archivos se intercambian directamente entre pares.



3.2. Componentes

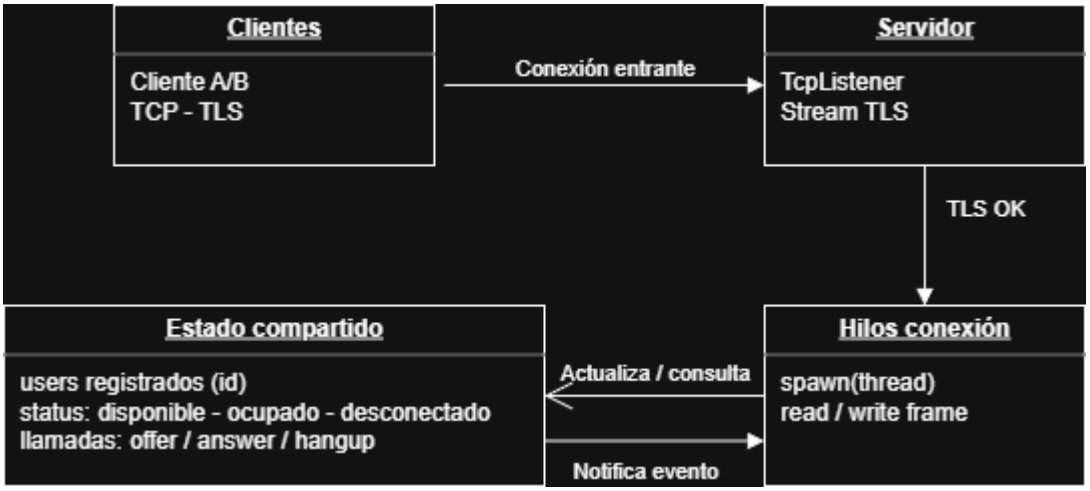
3.2.1. Signaling Server (modelo Cliente–Servidor)

El servidor de señalización se implementa como un servicio TLS que:

- Escucha conexiones entrantes (TCP).
- Realiza el handshake TLS mediante rustls.
- Maneja cada conexión en un hilo independiente.
- Mantiene un estado global con usuarios registrados, estado (Available/Busy/Disconnected) y eventos de llamada.

El protocolo de aplicación utiliza mensajes serializados en JSON con framing explícito:

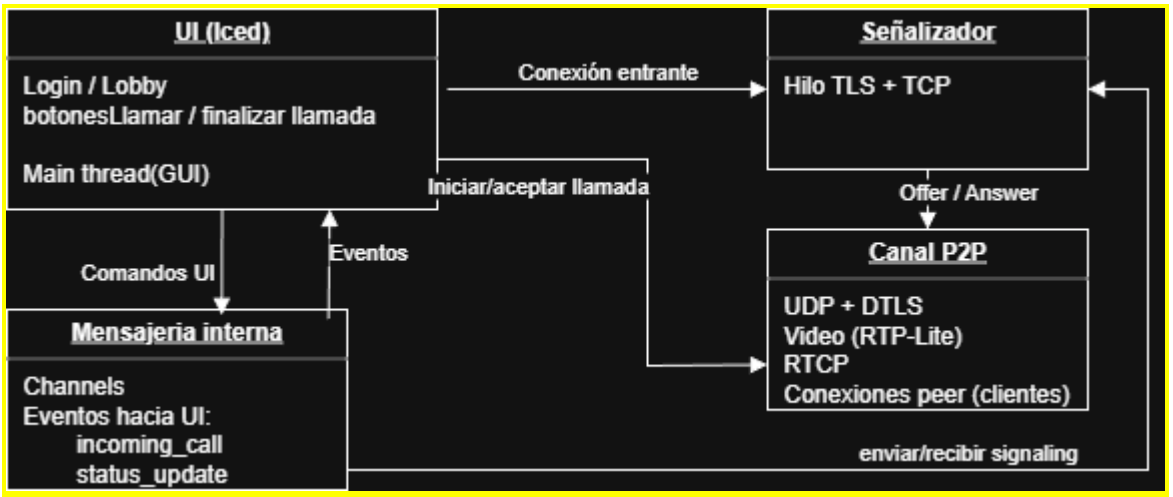
- **[4 bytes big-endian: longitud] + [N bytes: JSON]**
  Esto permite lecturas robustas sobre un stream TCP, evitando ambigüedad de delimitación.



3.2.2. Cliente (UI + Media + Red)

Cada cliente está compuesto por una aplicación con UI (Iced) más un conjunto de módulos de media y red. La idea central es separar:

- **UI / lógica de interacción** (login, lobby, botones de llamada).
- **Señalización** (hilo TLS en background).
- **Canal seguro P2P** (DTLS sobre UDP para video+SCTP+RTCP).
- **Pipelines de media** (GStreamer para captura/encode/decode/render).



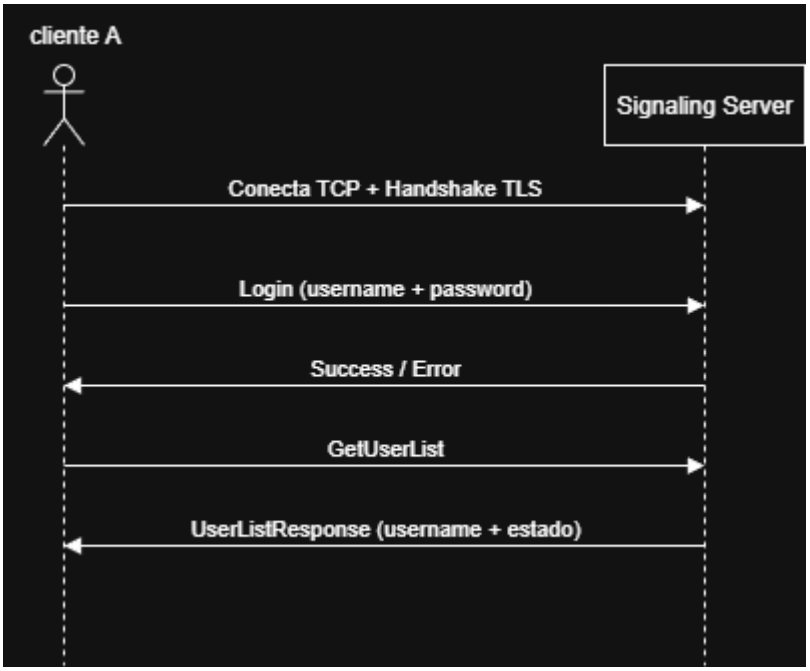
3.3. flujo de ejecución

El flujo de ejecución sigue los siguientes pasos:

A. UI (Iced) + canales internos

La UI se ejecuta como una aplicación unificada (evitando recreación de ventana) y se comunica con el backend mediante canales (mpsc):

- Envía comandos: login/register, refresh de lobby, call request, call accept/reject, enviar archivo, etc.
- Recibe eventos: resultado de autenticación, llamada entrante, llamada aceptada/rechazada, fin de llamada, etc.
- Recibe frames de video ya decodificados para render.



B. Señalización (TLS) en background

El cliente mantiene un hilo dedicado que:

- Abre conexión TLS al Signaling Server.
- Procesa autenticación y luego itera atendiendo comandos de UI y mensajes entrantes del servidor.
- Cuando una llamada se acepta, determina el rol (caller/callee) y entrega al subsistema DTLS el fingerprint remoto mediante un canal interno.

C. ICE-lite / transporte UDP base (socket único)

El cliente crea un socket UDP ligado al puerto de video. A partir de ese socket:

- Se establece DTLS con el peer usando udp_dtls (OpenSSL por debajo).
- Se usa el mismo canal para transportar video, RTCP y SCTP (multiplexado).

Se incluye lógica de STUN (si está disponible) para descubrir dirección pública, como base para escenarios con NAT.

D. Canal seguro: DTLS + verificación de fingerprint

Luego de la aceptación de llamada:

- Un hilo “DTLS Manager” ejecuta el handshake DTLS sobre el UDP base.
- Se aplica una verificación de fingerprint (SHA-256) del certificado remoto.
- El resultado se publica como un stream DTLS compartido (Arc<Mutex<Option<...>>>) usado por los hilos TX/RX y SCTP/RTCP.

E. Video: VP8 + (RTP-Lite) + Jitter Buffer

El video sigue un pipeline lógico:

- Captura (GStreamer): frames crudos
- Encode VP8: bytes comprimidos
- Packetización “RTP-Lite”: fragmentación en paquetes
- Envío por DTLS
- Recepción por DTLS: reensamble de paquetes
- JitterBuffer (reordenamiento + delay objetivo)
- Decode VP8 (GStreamer): frames para UI

F. Cierre ordenado (shutdown)

El diseño incorpora una bandera global de shutdown (AtomicBool) para asegurar que al cerrar la UI:

- Terminen los loops de señalización/auth.
- Se detienen los hilos TX/RX/RTCP.

- No queden threads bloqueados impidiendo finalizar el proceso.

4. Diseño e Implementación

Este capítulo describe cómo se materializa la arquitectura propuesta: señalización segura (cliente–servidor), establecimiento del canal seguro P2P con DTLS, transmisión de video sobre ese canal, y mecanismos de sincronización para mitigar jitter y reordenamiento.

4.1. Señalización

4.1.1. Objetivo

La señalización cumple dos funciones principales:

- ➢ **Control de sesión** (login/registro, lobby, request/accept/reject/end).
- ➢ **Bootstrapping de seguridad P2P**, intercambiando la información mínima para que los pares puedan abrir un canal cifrado sin depender del servidor para transportar media.

En esta implementación, la seguridad se compone de **dos capas**:

- **TLS (cliente – servidor)**: protege credenciales y mensajes de control durante la señalización.

- **DTLS (cliente – cliente)**: protege el canal P2P que transporta el **video**

4.1.2. Protocolo de aplicación (sobre TLS)

Los mensajes de señalización se serializan en **JSON** y se envían con un framing explícito:

4 bytes big-endian: longitud] + [N bytes: JSON]

Esto permite recuperar mensajes completos sobre TCP sin ambigüedades (evitando “pegar” mensajes o cortar lecturas).

El protocolo define, entre otros, los siguientes tipos (relevantes a llamada):

- `CallRequest{ callee, fingerprint }`
- `CallIncoming{ caller, fingerprint }`
- `CallAccept{ caller, fingerprint }`
- `CallReject{ caller }`

- `CallEnd`

Además existen `Offer/Answer/IceCandidate` como extensión, pero **la versión entregada establece la llamada práctica con CallRequest/CallAccept + fingerprint**, y los parámetros de red provienen del `.config`.

Secuencia completa de establecimiento

El resumen del flujo end-to-end:

➔ El cliente abre una conexión **TLS** hacia el servidor, y ejecuta **Login/Register**.
➔ Ya autenticado, consulta el lobby con `GetUserList`.
➔ Al iniciar una llamada, el caller envía `CallRequest{callee, fingerprint_A}`.
➔ El servidor notifica al callee con `CallIncoming{caller, fingerprint_A}`.
➔ Si el callee acepta, responde `CallAccept{caller, fingerprint_B}`.
➔ El servidor reenvía al caller `CallAccept{caller, fingerprint_B}`.

A partir de ese punto, ambos clientes habilitan el establecimiento del canal P2P:

- Cada cliente entrega al **hilo DTLS Manager** un par `(rol, fingerprint_remoto)`:
    - **Caller:** `(true, fingerprint_B)`
    - **Callee:** `(false, fingerprint_A)`

4.1.3. Parámetros de red (UDP base)

En la entrega actual, el socket UDP base del canal P2P se liga al **puerto de video** leído del `.config` (formato SDP-like)

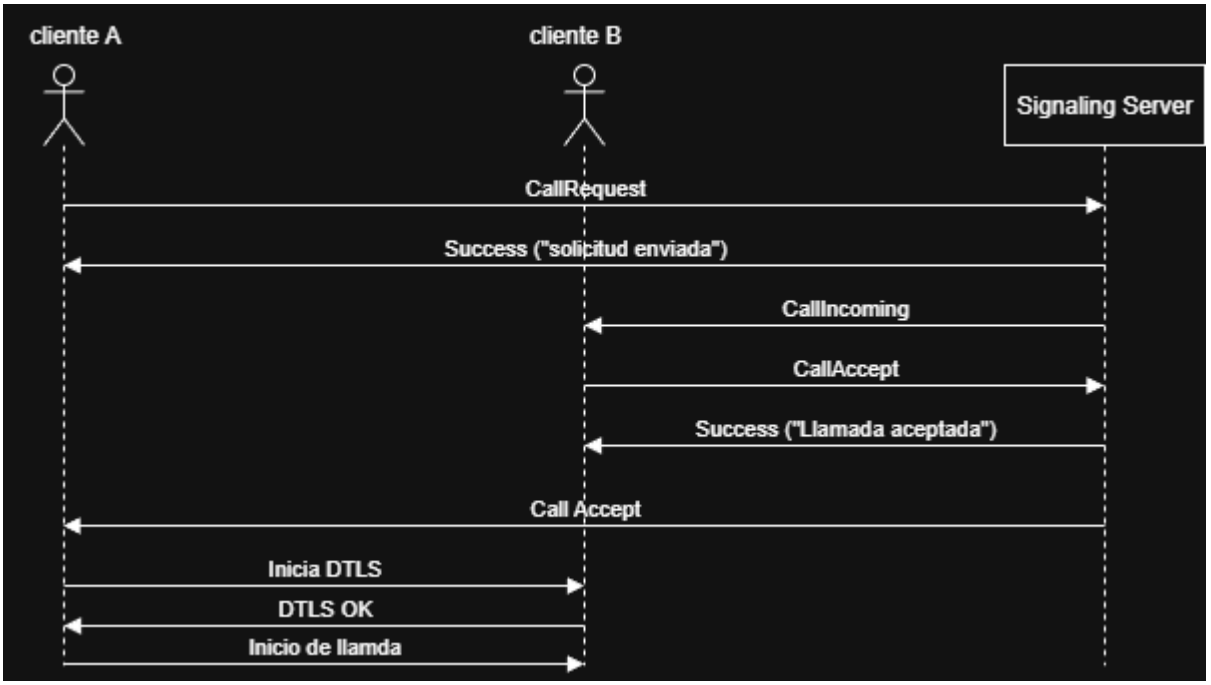
4.1.4. Handshake DTLS + verificación de fingerprint

Cada cliente genera en tiempo de ejecución una **identidad autofirmada** (certificado X.509) y computa su **fingerprint SHA-256**. Ese fingerprint es el que viaja por señalización en `CallRequest/CallAccept`.

Luego, en el DTLS Manager:

- El socket UDP se pasa momentáneamente a **modo bloqueante** y se setea un **timeout** corto para permitir retransmisiones DTLS.
- Se ejecuta el handshake:
    - Caller: `DtlsConnector.connect(...)`
    - Callee: `DtlsAcceptor.accept(...)` (configurando el certificado local)
- Al finalizar, se lee el certificado remoto y se calcula su digest SHA-256.
  Condición de seguridad: `fingerprint_cert_remoto == fingerprint_remoto_recibido_por_signaling`
  Si no coincide, la conexión se rechaza (mitigación de ataque MitM, asumiendo que la señalización TLS es confiable).
- Finalmente se vuelve el UDP a **modo no bloqueante** para la operación normal.

El resultado del handshake (un `DtlsStream`) queda compartido mediante `Arc<Mutex<Option<DtlsStream>>>` para los hilos de TX/RX.



4.2. Transmisión de video seguro

4.2.1. Objetivo

Transmitir video P2P con:

- **Baja latencia**

- **Cifrado** (DTLS)
- Un formato de transporte simple y controlado por el proyecto (RTP-Lite)

4.2.2. Pipeline: captura/encode - packetización - DTLS

El video se captura y codifica con GStreamer a **VP8**. El encoder entrega `EncodedFrame` (bytes VP8) a un canal interno consumido por un **hilo TX de video**.

En el TX de video se implementa RTP-Lite, una cabecera mínima de 11 bytes:

- marker (1 bit) + payload_type (7 bits, se usa 96)
- sequence_number (u16)
- timestamp (u32, reloj 90 kHz)
- ssrc (u32)

Cada frame VP8 se fragmenta en múltiples paquetes con MTU ≈ 1200. El último fragmento se marca con `marker=1`.
El timestamp se incrementa en 3000 por frame (coherente con 30 FPS en un reloj RTP de 90 kHz: 90.000/30 = 3000).

Los paquetes resultantes se envían cifrados usando `stream.write_all(&pkt)` sobre el `DtlsStream`.

4.2.3. Recepción: DTLS - demux - reensamble - jitter buffer

El hilo RX lee bytes desde el `DtlsStream` y clasifica el contenido:

- Si el payload luce como **RtpLite** (payload_type=96 + parse válido), se procesa como video.
- Si no, el canal está preparado para otros tipos multiplexados (tratados en §5).

Para video:

➔ Se parsea el `RtpLitePacket`.
➔ Se actualizan estadísticas de recepción (`RtpRecvStats`).
➔ Se reensambla el frame VP8 con `RtpLiteReassembler`:
  ◆ Agrupa fragmentos por **timestamp**
  ◆ Cuando llega un paquete con `marker=1`, concatena fragmentos por `sequence_number` y produce el frame completo.
➔ El frame completo se encola en el **JitterBuffer** con:
  ◆ `timestamp_90khz`
  ◆ `arrival = Instant::now()`

- `data = frame VP8`

4.2.4. Control adaptativo simple

Como mecanismo simple de adaptación ante degradación de red, el TX evalúa `fraction_lost` (derivado de estadísticas de recepción) y aplica un “drop factor”:

- Si `loss > 10%`, envía **1 de cada 2 frames** (reduce el bitrate efectivo).
- Si no, envía todos los frames.

Este mecanismo es deliberadamente simple, pero permite mantener fluidez cuando la red introduce pérdida sostenida.

4.2.5. Consideraciones de concurrencia

El `DtlsStream` se comparte entre hilos mediante `Mutex`. Por eso:

- RX y TX compiten por el lock.
- En RX, si la lectura devuelve `WouldBlock`, se hace un sleep corto para **ceder CPU** y evitar starvation del TX.

4.3. Sincronización y Jitter Buffer

4.3.1. Problema a Resolver

En redes reales (y especialmente en escenarios emulados), los paquetes pueden:

- llegar **con variación de retardo** (jitter),
- llegar **reordenados**,
- o llegar **tarde** respecto del ritmo de reproducción.

Si el decoder consume frames “apenas llegan”, la reproducción se vuelve inestable (saltos, congelamientos cortos, bursts).

4.3.2. Estrategia implementada

Se implementa un **JitterBuffer** dedicado para frames VP8 ya reensamblados, con dos parámetros:

- `target_delay`: cuánto tiempo se acumula antes de empezar a reproducir (en la configuración actual: **80 ms**)
- `max_delay`: cuánto se tolera que un frame quede “viejo” desde que arribó (en la configuración actual: **200 ms**)

Estructura interna:

- ➔ Se guarda cada frame en un heap ordenado por **timestamp** (permite reordenamiento).
- ➔ Al primer intento de reproducción se inicializa `start_playout`.
- ➔ Hasta alcanzar `target_delay`, **no se libera ningún frame** (buffer inicial).
- ➔ Una vez en régimen:
  - ◆ Se libera el frame más antiguo por timestamp.
  - ◆ Si un frame excede `max_delay` desde su llegada, se descarta (evita que "arrastre" latencia).

4.3.3. Consumidor de Jitter

Un hilo "consumidor" ejecuta en loop:

- `pop_ready(Instant::now())`
- si devuelve un frame: lo envía al decoder por canal interno
- si no hay frame listo: duerme muy poco para no ocupar CPU

Esto desacopla el ritmo de llegada del ritmo de decodificación/render.

5. Diseño e implementación del agregado final

5.1. Transferencia de Archivos

5.1.1. Objetivo

Agregar envío de archivos **sin pasar por el servidor** (P2P), reutilizando el canal seguro ya establecido entre clientes. Para esto se implementa **SCTP**, que aporta confiabilidad, ordenamiento y control de congestión a nivel transporte, sin necesidad de reimplementar esas garantías a mano.

5.1.2. Diseño: SCTP sobre DTLS

La implementación utiliza un **único socket UDP base** (el del video), sobre el cual corre DTLS. Encima del `DtlsStream` se multiplexan:

- **Video** (RtpLite)
- **SCTP** (transferencia de archivos) *(y RTCP, documentado en el capítulo anterior)*

El punto crítico es el **demultiplexado en recepción**, resuelto dentro del thread RX que lee de DTLS:

- Si el primer byte sugiere **payload_type = 96** y el parseo de **RtpLite** es válido → se procesa como **video**.
- Si no es video y el tamaño es suficiente (mínimo) → se trata como **SCTP** y se entrega al endpoint SCTP.

5.1.3. Canal logico: Stream SCTP dedicado

Se reserva el **Stream ID = 1** de SCTP para archivos. Esto permite:

- Separar semánticamente “archivo” de otros posibles datos futuros (mensajes, control, etc.)
- Mantener independencia lógica aunque viajen por el mismo DTLS/UDP

5.1.4. Formato de aplicación del archivo

Sobre el stream SCTP se envía un payload con framing simple:

`[ContentLen: u32][NameLen: u16][Name][Content]`

Esto permite reconstrucción determinística del archivo en el receptor aun si el stream entrega datos en varias lecturas.

5.1.5. Envío

Cuando la UI solicita enviar un archivo:

- Se lee el archivo completo desde disco.
- Se arma el payload con el encabezado anterior.
- Se escribe al stream SCTP en **fragmentos** (`CHUNK_SIZE ≈ 2000 bytes`).
- Dado que `write_sctp()` puede escribir menos bytes que los pedidos, se implementa un **loop de “partial write”** hasta completar cada fragmento.
- Se llama `assoc.poll_transmit()` para obtener los datagramas listos a enviar y se transmiten por el canal seguro con `dtls.write_all(...)`.

5.1.6. Recepción

La recepción se divide en dos responsabilidades:

- ➢ **Alimentación del stack SCTP (demux + endpoint.handle)**
  El thread RX que lee DTLS detecta datagramas SCTP y los inyecta al endpoint (ej. `endpoint.handle(...)`). Si el endpoint produce eventos (nueva asociación o eventos de una asociación), se procesan con `assoc.handle_event(...)`.
  Cuando hay respuestas inmediatas (handshake/acks internos), se obtienen con `assoc.poll_transmit()` y se envían por DTLS.
- ➢ **Consumo de datos de aplicación (poll + read_sctp)**
  Un thread dedicado hace `assoc.poll()` y, ante `DatagramReceived`, lee del **Stream 1** con `read_sctp()`. Los bytes se van acumulando en un buffer hasta completar el mensaje según el header `[ContentLen][NameLen]`.
  Cuando el mensaje está completo, se persiste en disco como:
  `received_<filename>`

5.2. Audio

5.2.1. Objetivo

Agregar audio de manera simple y desacoplada, evitando interferir con el canal DTLS de video y su timing, y permitiendo depuración/validación independiente.

5.2.2. Diseño: Opus/RTP por UDP

El audio se envía por **UDP directo**, usando un **puerto independiente** derivado por convención:

`puerto_audio = puerto_video + 2`

Esto evita colisiones con los paquetes del canal DTLS (video/SCTP) y permite operar con sockets y buffers separados.

5.2.3. Pipeline Gstreamer

- **Transmisión (TX):**
  - `autoaudiosrc - opusenc - rtpopuspay - appsink`
  - El `appsink` entrega paquetes RTP (Opus) que se envían con `udp.send_to(...)`.
- **Recepción (RX):**
  - `udp.recv_from(...) - appsrc - rtpopusdepay - opusdec - autoaudiosink`

6. Modelo de concurrencia

El cliente está diseñado como una aplicación **multihilo**, separando responsabilidades para:

- mantener la **UI reactiva** (sin bloqueos por I/O),
- aislar tareas de **tiempo real** (video/audio),
- y encapsular el establecimiento/uso del **canal seguro DTLS**.

6.1. Hilo Principal (Main / GUI)

- **Origen:** `fn main` y luego `run_media_client` → `run_unified_app`.
- **Responsabilidad:** ejecutar el loop de la UI, crear canales y disparar la infraestructura de threads.
- **Tareas clave:**
  - Cargar configuración (SDP-like) y preparar parámetros (fps, puertos, RTCP).
  - Inicializar estado compartido para autenticación y señalización (`pending_auth`, `auth_result`, `signaling_response`).
  - Lanzar el **hilo de señalización** (auth_thread) en background.
  - Entrar en `run_unified_app`, que mantiene la ventana activa y traduce eventos de UI a comandos.

6.2. Hilo de Señalización (`auth_thread`)

- **Responsabilidad:** mantener comunicación **TLS** persistente con el Signaling Server.
- **Entradas/Salidas:**

    - Entrada: `mpsc::Receiver<SignalingCommand>` (comandos desde la UI).
    - Salida: `Arc<Mutex<SignalingResponse>>` (respuesta para que la UI la levante).
      Salida adicional: `role_tx` (envía rol y fingerprint remoto al DTLS Manager cuando una llamada queda aceptada).
- **Tareas:**
    - Handshake TLS, login/register, y manejo del protocolo JSON con framing.
    - Procesar comandos de lobby (`GetUserList`, etc.).
    - Manejar eventos asíncronos relevantes a llamadas (`CallIncoming`, `CallAccept`, etc.).
    - Entregar a DTLS el par **(rol, fingerprint remoto)** mediante `role_tx`.
- **Ciclo de vida:** corre en loop hasta detectar cierre global. Para evitar "hilos colgados", el cierre de la UI debe activar una bandera de shutdown global para que este hilo abandone el loop y permita el `join.`

6.3. Hilo Gestor de DTLS (`DTLS Manager`)

- **Responsabilidad:** establecer el canal **DTLS P2P** de forma bloqueante y segura, y publicar el stream listo para los hilos de media/datos.
- **Tareas:**
    - Espera bloqueante a que la señalización confirme llamada (`role_rx.recv()`).
    - Configura el UDP temporalmente como **bloqueante** con timeout para permitir retransmisiones de DTLS durante handshake.
    - Ejecuta handshake como:
        - caller → `connect`
        - callee → `accept`
    - **Verificación de seguridad:** compara el fingerprint SHA-256 del certificado remoto contra el fingerprint recibido por señalización.
    - Publica el resultado en `dtls_stream:`
      `Arc<Mutex<Option<DtlsStream>>>`.
      Inicializa estructuras de SCTP (endpoint/asociación) para habilitar datos multiplexados.
    - Levanta `running_call=true` para habilitar TX/RX de media.
- **Ciclo de vida:** se activa una vez por llamada (y/o por "inicio de sesión segura"), y luego deja a los demás threads operar sobre el `dtls_stream`.

6.4. Hilos de Video (Transporte DTLS)

En la implementación actual se utilizan tres threads principales para el plano de video

6.4.1. Hilo de Transmisión (`tx_thread`)

- **Entrada:** frames VP8 desde el encoder (canal `rx_enc` generado por GStreamer).
- **Tareas:**
    - Fragmentación + encapsulado en **RTP-Lite** (sequence, timestamp, marker, SSRC).
    - Control simple de carga: posibilidad de "dropear frames" cuando se detecta pérdida elevada (según estado de calidad).

- Envío cifrado: `stream.write_all(packet)` sobre el `DtlsStream`.
- Gating: si `running_call=false`, no envía (la llamada aún no está activa).

6.4.2. Hilo de Recepción (`rx_thread`)

- **Entrada:** bytes desencriptados leídos desde `DtlsStream`.
- **Tareas:**
  - **Demultiplexación:** decide si el datagrama es **Video (RTP-Lite)** o **Datos (SCTP)** inspeccionando el contenido (parse/validación).
  - Ruta video:
    - parse de RTP-Lite,
    - actualización de estadísticas,
    - reensamble por `timestamp` con `RtpLiteReassembler`,
    - inserción del frame VP8 completo en el **JitterBuffer**.
  - Ruta datos:
    - entrega datagramas al stack SCTP y procesa eventos.
  - Mantenimiento SCTP: ejecuta `poll_transmit` periódicamente para enviar heartbeats/SACKs cuando corresponda (a través del mismo DTLS).

6.4.3. Hilo Consumidor de Jitter Buffer (`jitter_thread`)

- **Entrada:** frames completos en JitterBuffer (reordenados y con delay objetivo).
- **Tareas:**
  - Extrae frames listos para reproducción (`pop_ready`).
  - Envía los frames VP8 al decoder/pipeline de visualización (vía canal interno).

***Nota*:** además de estos tres threads “de aplicación”, GStreamer opera con threads internos (captura/codec) y existen threads auxiliares para empujar buffers a `appsrc` en el decoder/display

6.5. Hilos de Datos (SCTP)

6.5.1. Hilo de Envío SCTP (`sctp_tx_thread`)

- **Entrada:** comandos desde UI (`mpsc::Receiver<SctpCommand>`), por ejemplo `SendFile(path)`.
- **Tareas:**
  - Leer archivo desde disco y construir payload con framing (metadatos + bytes).
  - Escribir en el **Stream SCTP** reservado para archivos (fragmentando y manejando “partial writes”).
  - Obtener datagramas de salida del stack (`assoc.poll_transmit`) y enviarlos por el canal seguro (`dtls.write_all`).
- **Recepción/poll SCTP (caller):** en la implementación actual existe además un thread dedicado a:
  - `assoc.poll()` para detectar `DatagramReceived`,
  - leer de streams (`read_sctp`),
  - reensamblar el contenido (según header),

- persistir `received_<filename>`,
  y, si corresponde, transmitir respuestas generadas por SCTP.

6.6. Hilos de Audio

El audio se implementa como agregado final independiente del canal DTLS, con **dos threads de red** y threads internos de GStreamer

6.6.1. Hilo TX Audio (`audio_tx_thread`)

- **Entrada:** paquetes Opus/RTP provenientes del encoder (canal `mpsc` generado por GStreamer).
- **Tareas:**
  - Si `running_call=true`, envía por UDP con `send_to` hacia `remote_audio = remote_video + 2`.
  - Si `shutdown=true`, abandona el loop.
  - Respeta `audio_muted` (mute a nivel de encoder).

6.6.2. Hilo RX Audio (`audio_rx_thread`)

- **Entrada:** UDP no bloqueante (`recv_from`).
- **Tareas:**
  - Si `running_call=true`, recibe paquetes Opus/RTP y los reenvía al player (`mpsc::Sender<Vec<u8>>`) para reproducción.
  - Sleep corto en `WouldBlock` para evitar consumo innecesario de CPU.
  - Sale por `shutdown=true`.

6.7. Hilos Auxiliares

Además de los threads principales, se crean threads auxiliares que cumplen funciones de soporte:

- **RTCP TX (`rtcp_tx_thread`)**: envía reportes periódicos a través del mismo `DtlsStream` (cuando está habilitado), útil para estadísticas y mantenimiento de sesión.
- **Stdin listener (mute)**: un loop bloqueante separado que lee `m`+Enter para alternar `audio_muted`, evitando bloquear la UI.
- **Threads auxiliares de media/GStreamer:**
  - Decoders con `appsrc`: threads que empujan buffers al pipeline (ej. VP8 decode y/o RGBA display).
  - Audio encoder/player: threads que mantienen vivo el pipeline leyendo el bus de GStreamer y/o alimentando `appsrc`.
  - Threads internos de GStreamer: captura, conversión y codec.

7. Conclusiones

En este proyecto se logró construir un sistema de comunicación en tiempo real con una arquitectura híbrida que separa de forma clara el **plano de control** del **plano de datos**. La decisión de mantener la señalización en un modelo Cliente–Servidor y transportar la media en P2P permitió cumplir el objetivo central: minimizar la carga del servidor y reducir la latencia, delegando el ancho de banda de video (y los datos agregados) directamente entre los participantes. El servidor queda así como una pieza de coordinación y estado, mientras que la transmisión efectiva sucede entre pares, lo que acerca el diseño a los principios de soluciones modernas de RTC.

Desde el punto de vista de seguridad, el resultado más relevante es que la transmisión de video se realizó sobre un canal **cifrado y autenticado** mediante DTLS, con un mecanismo de verificación de identidad basado en **fingerprints** intercambiados durante la señalización. Esto permitió que, una vez establecida la sesión por TLS con el servidor, los clientes pudieran crear un canal P2P seguro sin exponer el contenido multimedia en la red. Además, la integración de estadísticas y reportes de control sobre el mismo canal habilita una base sólida para evolucionar hacia mecanismos de adaptación y monitoreo.

En el plano multimedia, la implementación alcanzó una cadena funcional y estable de extremo a extremo: captura, codificación VP8, empaquetado (RTP-Lite), envío, recepción, reensamble y decodificación. En particular, la incorporación del **Jitter Buffer** resultó determinante para sostener una reproducción fluida bajo condiciones de red con variación de retardo y reordenamiento, desacoplando la llegada irregular de paquetes del ritmo de reproducción. Esta decisión mejoró perceptiblemente la estabilidad del video y mostró la importancia de introducir sincronización temporal incluso en prototipos académicos.

El agregado final de transferencia de archivos demostró que la arquitectura era extensible sin reestructurar el núcleo del sistema: al multiplexar SCTP sobre el canal DTLS ya existente se consiguió un mecanismo confiable y ordenado para transmitir datos P2P de manera segura, sin sumar nuevos sockets ni incrementar la complejidad de despliegue. Esta extensión validó que el enfoque de “un canal seguro compartido” es una base práctica para incorporar funcionalidades más allá del video, siempre que se implemente cuidadosamente el demultiplexado y la coordinación entre hilos.

A nivel de arquitectura, uno de las implementaciones más importantes del trabajo fue el modelo de **concurrencia**: se diseñó un cliente con múltiples hilos especializados (UI, señalización, gestor DTLS, transmisión/recepción de video, consumo de jitter, datos, audio y auxiliares) comunicados por canales y estado compartido mínimo. Este enfoque permitió mantener la interfaz reactiva, aislar tareas de tiempo real y evitar bloqueos prolongados. En la misma línea, el manejo de **cierre ordenado** mediante banderas de shutdown y la terminación explícita de loops y threads resolvió un problema típico en sistemas concurrentes: evitar que el proceso quede colgado al cerrar la UI.

El audio se integró como un subsistema independiente por UDP, priorizando simplicidad y aislamiento frente a riesgos de regresión sobre el canal DTLS ya estabilizado. La negociación P2P se apoya en convenciones prácticas de puertos y roles para facilitar pruebas, y el demultiplexado de flujos en recepción se resuelve con validación heurística de

paquetes. Estas decisiones fueron adecuadas para completar la funcionalidad requerida y entregar un sistema operable.